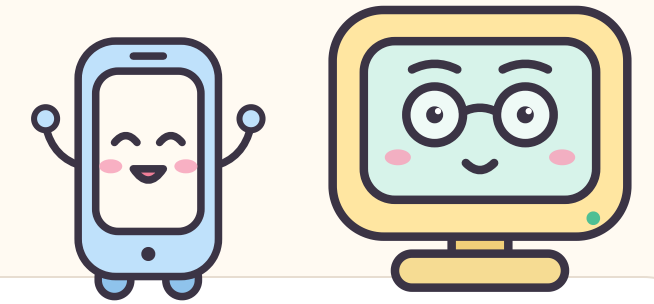




第 0 集 · 约 20 分钟

序章：AI 都会写代码了，为什么还要学编程

不喊口号，只看证据：正面的、反面的都看



从零写出 LanShare

这一集你会学到

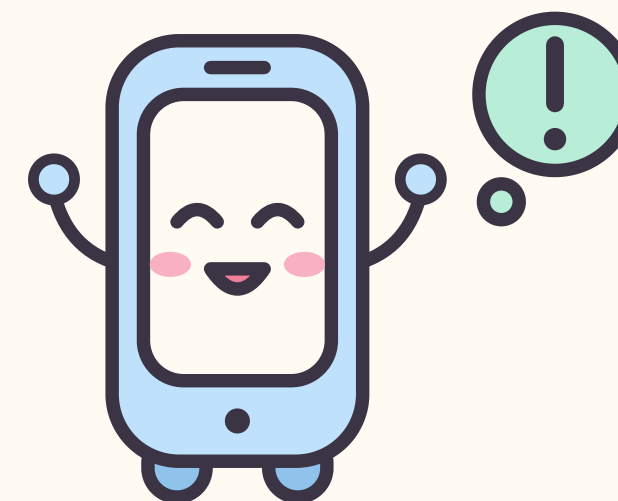
1. AI 写代码到底让人快了多少：哪些实验说快，哪些说慢
2. AI 写的代码在安全上的真实表现
3. 学习的时候怎么用 AI，决定了你学到多少
4. 为什么写代码越来越便宜，懂代码的人反而越来越值钱
5. 怎么读一个数字：样本多大、测的是什么、谁做的研究



一个绕不开的问题

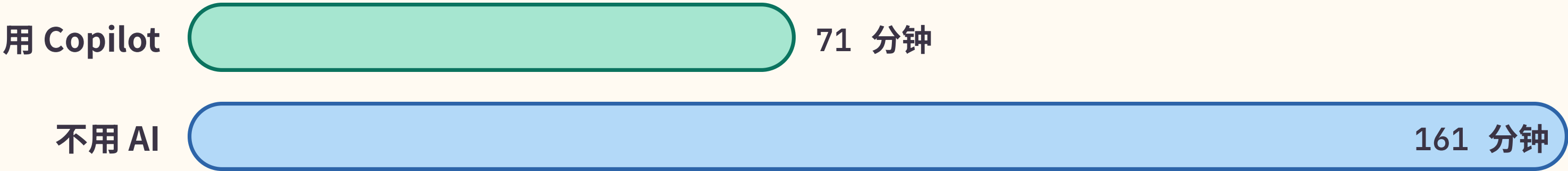
AI 都能写代码了，还学编程？

这个问题值得认真回答。答案不在口号里，在实验数据里。





证据一：小任务上，AI 确实快



用 JavaScript 写一个 HTTP 服务器，完成者的平均用时。Peng 等，2023（实验在 2022 年做，95 名自由职业者）



证据二：在真实的公司里，效果是正的，但没那么夸张

● 三家公司、近 5000 名开发者：真正用上的人，每周完成的任务多了约 26%

误差范围很大，新手受益更多，资深开发者的提升不显著（Cui 等，2025）

● 谷歌内部实验：用 AI 的组平均用时少约 16%

控制其他因素后估计约 21%，但统计上不显著（Paradis 等，2024）



证据三：在复杂的真实项目里，结果曾经相反

谁说的	用 AI 之后的用时
开发者事前预计	快 24%
做完以后自己觉得	快 20%
实际测量	慢 19%



但是，要公平地看这个实验

- 样本只有 16 人，用的是 2025 年初的工具
- METR 2026 年的更新：现在的工具“很可能”已经能提速，但数据只是很弱的证据
- 结论：工具在快速进步；但快不快，取决于任务，也取决于用的人



证据四：AI 写的代码，安全吗



不同类型漏洞的安全通过率（四年来 100 多个模型的累计口径）。Veracode 2026 年报告，专门设计的安全敏感任务



证据五：更隐蔽的问题

用 AI 的人，更容易觉得自己的代码是安全的

斯坦福的用户实验，47 人，多数是学生，2022 年的模型（Perry 等，2023）

AI 会编造不存在的软件包名，坏人可以抢先注册来投毒

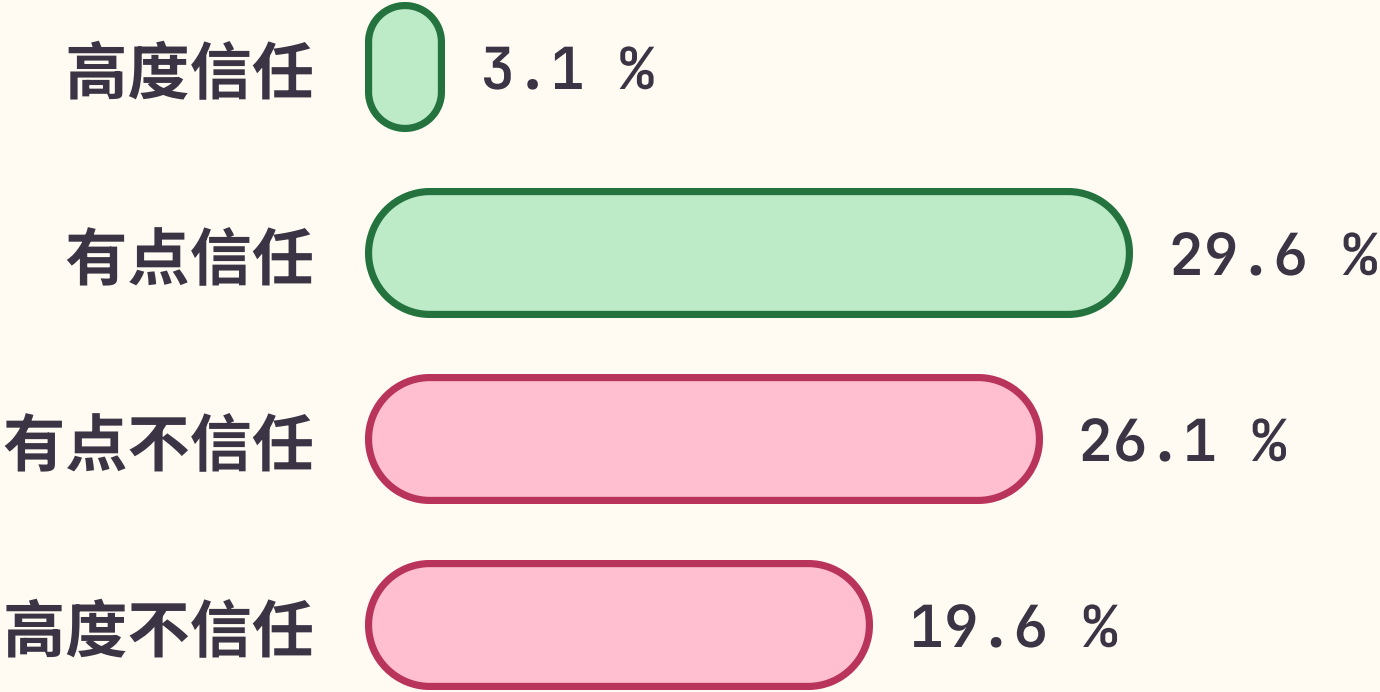
2024 年：开源模型约 22%，商业模型约 5%；2026 年复测前沿模型仍约 5%（预印本）

开发者最大的抱怨：“差一点就对了”

Stack Overflow 2025 年调查，66% 选了这一项



程序员们自己怎么看



“你信任 AI 工具输出的准确性吗？” 其余约 22% 选了中立。Stack Overflow 2025 年开发者调查，约 3.3 万人回答



证据六：瓶颈搬家了

谷歌 DORA 报告：AI 是“放大器”

好团队更好，问题团队的问题更突出（2025）

写代码变快了，审查跟不上了

一项企业研究：两年里人均 PR 翻倍，经过人工审查的比例从 89% 降到 68%（预印本，2026）

结论：写代码变便宜了，判断代码变贵了



对学习者的最重要的一个实验

自己动手写 67 分

可以用 AI 助手 50 分

学一个新的 Python 库之后马上测验的平均分。可以用 AI 的一组怎么用由各人自己决定。Anthropic，2026，52 名工程师，随机对照



同样用 AI，有人学会了，有人没学会

- 得分低的用法：整段交给 AI 写；越来越依赖；靠 AI 反复调试
- 得分高的用法：生成代码后追问原理；要代码也要解释；只问概念
- 这门课的做法：每一行都亲手写，每一行都讲为什么



业界大佬怎么说：并不一致

- 黄仁勋，2024 年 2 月：“我们的工作，是创造一种计算技术，让谁都不需要编程。”

他建议把精力放在生物、教育、制造、农业等领域专长上（据第三方转录）

- 吴恩达，2025 年 3 月：“劝人别学编程，将被看作有史以来最糟糕的职业建议之一。”

编程变容易时，学的人应该更多；懂软件的人能更精确地指挥 AI

- Anthropic CEO，2025 年 3 月：AI 会写几乎所有代码，但程序员仍要定需求、做设计、保证安全

“90% 的代码由 AI 写”是否兑现，至今有争议

- AWS CEO，2025 年 8 月：用 AI 取代初级员工，是“我听过最蠢的事之一”

“十年以后，没有一个人学会过任何东西，那怎么办？”



就业：冲击先落在入门岗位

● 美国 22 到 25 岁的软件开发者，人数比 2022 年底的峰值少了近 20%

截至 2025 年 9 月，ADP 薪资数据（斯坦福数字经济实验室）。资深员工没有类似下降

● 研究者强调：这是描述性事实，还不能证明是 AI 造成的

耶鲁 2025 年 10 月的研究：ChatGPT 发布后的 33 个月里，整体劳动市场没有出现可见的冲击

● 能读出的信号：入门的门槛变高了



这个项目里的“差一点就对了”

○ Windows 上，Rust 的改名函数会悄悄覆盖同名文件

编译通过、测试通过；Python 的同名函数在 Windows 上会报错，Rust 的不会

○ 一个下划线，让单实例锁当场失效

let 下划线等于 guard 会立刻释放；必须给它起个名字

○ 开发版的加密，每秒只有 8 兆字节

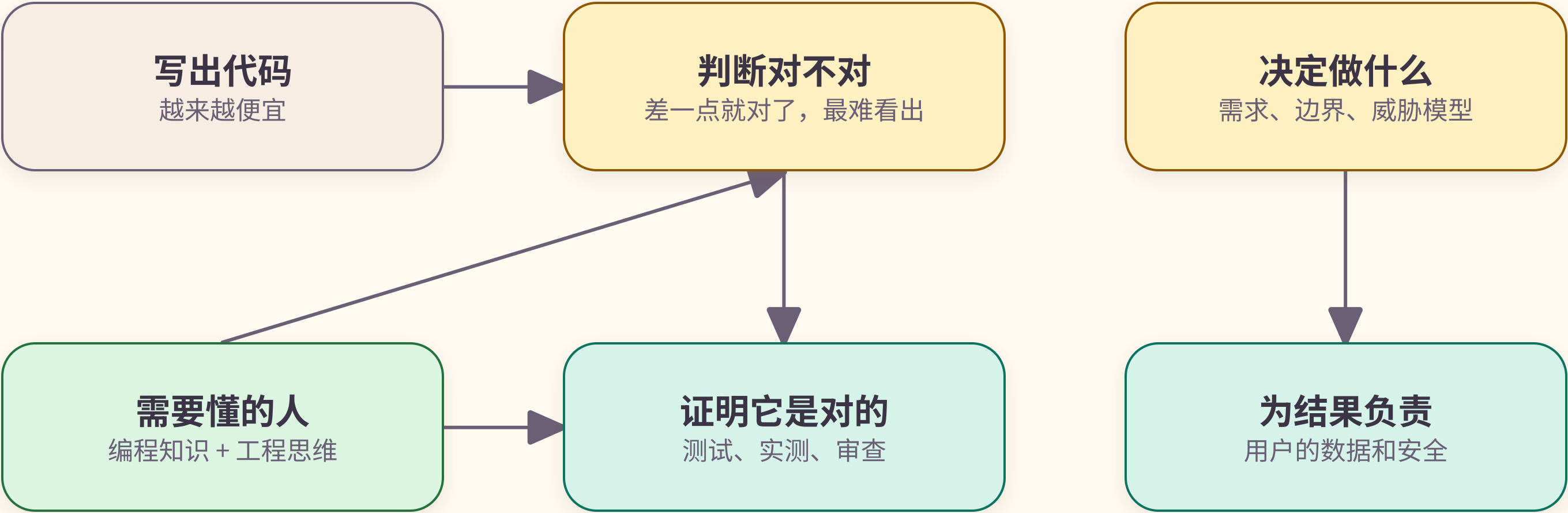
泛型代码在调用方编译；只优化依赖库不够

○ 第一版安全修复，自己又带来了新漏洞

新加的连接上限不分来源，局域网一台机器就能占满



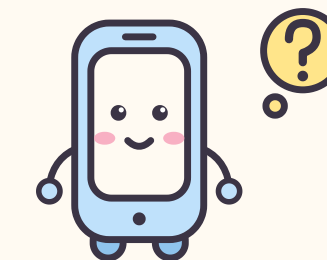
为什么懂的人更值钱





所以这门课会这样教

- 每一行代码亲手写，每一行讲为什么
- 每一道防线先写测试，证明攻击真的被挡住
- 每一个数字都来自实测，说清楚在什么条件下测的
- 说清楚它防什么，也说清楚它不防什么



选择题

METR 2025 年初的实验里，资深开发者用 AI 之后，实际用时和他们自己的感觉分别是怎样的？

A 实际快了约 20%，自己也觉得快了

B 实际慢了约 19%，自己却觉得快了约 20%

C 用不用 AI 没有任何区别



选择题

METR 2025 年初的实验里，资深开发者用 AI 之后，实际用时和他们自己的感觉分别是怎样的？

A 实际快了约 20%，自己也觉得快了

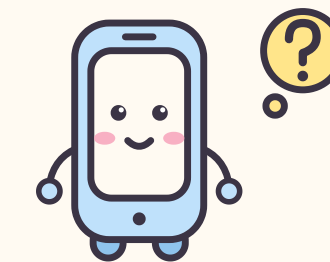
他们自己确实觉得快了约 20%，但实际测量是慢了。

C 用不用 AI 没有任何区别

测出来有差别，用 AI 时用时更长。

B 实际慢了约 19%，自己却觉得快了约 20%

这就是这个实验最值得记住的地方：人对 AI 帮了自己多少的感觉，并不可靠。当然，它只有 16 人，用的是 2025 年初的工具。



选择题

网上常说“AI 生成的代码有 45% 存在安全漏洞”，这个 45% 原本指的是什么？

- A 所有 AI 生成的代码里，45% 有漏洞
- B 在 80 个专门设计、有安全和不安全两种写法的任务里，AI 约 45% 的情况选了不安全的写法
- C 45% 的程序员不信任 AI



选择题

网上常说“AI 生成的代码有 45% 存在安全漏洞”，这个 45% 原本指的是什么？

A 所有 AI 生成的代码里，45% 有漏洞

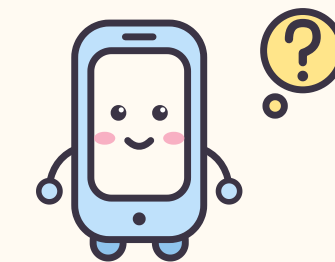
这是最常见的误传。原报告没有统计所有 AI 代码。

C 45% 的程序员不信任 AI

那是另一个调查的另一个数字，2025 年是 46% 不信任 AI 输出的准确性。

B 在 80 个专门设计、有安全和不安全两种写法的任务里，AI 约 45% 的情况选了不安全的写法

读一个数字，先看它测的是什么。这是专门挑出来的安全敏感任务，只覆盖 4 类漏洞，用静态分析判定。



选择题

Anthropic 的实验里，同样用 AI，测验得分高的人是怎么用的？

A 让 AI 写出完整代码，再仔细抄一遍

B 生成代码之后追问原理，或者只问概念、代码自己写

C 出了错就交给 AI 反复调试，直到能跑



选择题

Anthropic 的实验里，同样用 AI，测验得分高的人是怎么用的？

A 让 AI 写出完整代码，再仔细抄一遍

整段交给 AI 写，恰恰是得分低的用法之一。

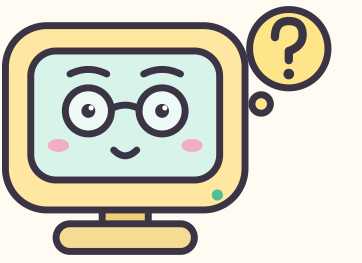
C 出了错就交给 AI 反复调试，直到能跑

靠 AI 反复调试，也是得分低的用法之一。

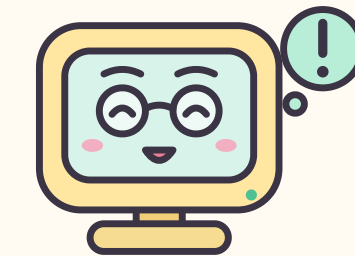
B 生成代码之后追问原理，或者只问概念、代码自己写

区别不在于用不用 AI，而在于拿 AI 抄答案，还是拿 AI 问为什么。

想一想



如果有一天，AI 写出的代码真的比你写的还好，你学编程还有什么意义？



想一想

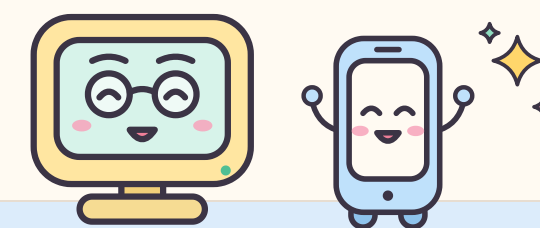
如果有一天，AI 写出的代码真的比你写的还好，你学编程还有什么意义？

你得先懂，才能判断它是不是真的更好。决定做什么、防什么、不防什么，证明它是对的，为结果负责，这些都需要懂的人。就像有了计算器，我们仍然要学数学：不是为了比计算器算得快，而是为了知道该算什么、算出来的结果对不对。



这一集我们看到

- ✓ 小任务上，AI 能让人快很多；复杂的真实项目里，结果曾经相反，而且人的感觉不可靠
- ✓ AI 写的代码在安全上仍然常出问题，用 AI 的人还更容易过度自信
- ✓ 学新东西时用 AI，测验平均分反而更低；关键在于拿 AI 抄答案，还是拿 AI 问为什么
- ✓ 写代码变便宜了，判断、设计、验证、负责变得更重要
- ✓ 读任何一个数字，先问：样本多大、测的是什么、谁做的研究



下一集

全课完结，感谢收看